BEGINNER TO INTERMEDIATE

Dart Programming Complete Topic Notes

বাংলা ব্যাখ্যা • Working code examples • Expected output • Key takeaways

Topics: Interface, Future, async/await, Stream basics, asynchronous exception handling, null safety, exception handling, generics, enums এবং extensions।

Prepared: October 2026 | Dart 3-style examples

সূচিপত্র

1. Interface
2. Future
3. async / await
4. Stream basics
5. Asynchronous exception handling
6. Null safety
7. Exception handling
8. Generics
9. Enums
10. Extensions

কীভাবে এই নোট ব্যবহার করবে

প্রতিটি টপিকের ব্যাখ্যা পড়ো, কোডটি **main.dart** ফাইলে লিখে **dart run main.dart** কমান্ডে চালাও, তারপর নিজে পরিবর্তন করে ফলাফল দেখো। প্রতিটি উদাহরণ আলাদাভাবে চালানোর জন্য লেখা হয়েছে।

TOPIC 01 / 10

Interface

Interface হলো এমন একটি contract, যেখানে কোনো class কী কী method/property দেবে তা নির্ধারিত হয়। Dart-এ আলাদা `interface` keyword দিয়ে সাধারণ interface declare করা হয় না; `class` বা `abstract interface class` ব্যবহার করা যায়। `implements` করলে প্রয়োজনীয় সব member implement করতে হয়।

Code Example

```
abstract interface class Printable {
  void printInfo();
}

class Student implements Printable {
  final String name;
  Student(this.name);

  @override
  void printInfo() => print('Student: $name');
}

void main() {
  Student('Rafi').printInfo();
}
```

Expected Output

```
Student: Rafi
```

মনে রাখবে: `extends` দিয়ে implementation inherit করা যায়; `implements` কেবল contract মেনে চলার বাধ্যবাধকতা তৈরি করে।

TOPIC 02 / 10

Future

`Future<T>` হলো এমন একটি value যা এখনো প্রস্তুত নয়, কিন্তু পরে সফলভাবে `T` value অথবা error নিয়ে complete হবে। Network request, file read, delay ইত্যাদিতে ব্যবহৃত হয়।

Code Example

```
Future<String> fetchName() async {
  await Future.delayed(Duration(seconds: 1));
  return 'Dart';
}

Future<void> main() async {
  print('Loading...');
  final name = await fetchName();
  print(name);
}
```

Expected Output

```
Loading...
Dart
```

মনে রাখবে: Future একবার complete হয়। Future তৈরি হওয়া মানেই সব কাজ আলাদা thread-এ চলছে—এমন নয়।

TOPIC 03 / 10

async / await

`async` দিয়ে asynchronous function লেখা হয়। `await` একটি Future-এর completion না হওয়া পর্যন্ত ওই async function-এর পরবর্তী অংশ স্থগিত রাখে; পুরো application block করে না।

Code Example

```
Future<int> addLater(int a, int b) async {
  await Future.delayed(Duration(milliseconds: 200));
  return a + b;
}

Future<void> main() async {
  final total = await addLater(5, 7);
  print('Total: $total');
}
```

Expected Output

```
Total: 12
```

মনে রাখবে: `await` সাধারণত `async` function-এর ভিতর ব্যবহার করবে। Independent Future থাকলে প্রয়োজনে `Future.wait` দিয়ে একসাথে অপেক্ষা করা যায়।

TOPIC 04 / 10

Stream basics

`Stream<T>` সময়ের সাথে একাধিক event দেয়। `Future` যেখানে একবার result দেয়, `Stream` সেখানে শূন্য, একটি বা অনেক value/error emit করতে পারে। `await for` অথবা `listen()` দিয়ে event consume করা যায়।

Code Example

```
Stream<int> countNumbers() async* {
  for (var i = 1; i <= 3; i++) {
    await Future.delayed(Duration(milliseconds: 100));
    yield i;
  }
}

Future<void> main() async {
  await for (final number in countNumbers()) {
    print(number);
  }
}
```

Expected Output

```
1
2
3
```

মনে রাখবে: `async*` ও `yield` দিয়ে stream তৈরি হয়। `listen()` একটি StreamSubscription দেয়, যা প্রয়োজন হলে cancel করা যায়।

TOPIC 05 / 10

Asynchronous exception handling

Asynchronous কাজ fail করলে `Future` error দিয়ে complete হতে পারে। `await` ব্যবহার করলে সেই error-কে সাধারণ `try/catch` দিয়ে handle করা যায়। Stream-এর ক্ষেত্রেও error event handle করতে হয়।

Code Example

```
Future<String> loadData() async {
  await Future.delayed(Duration(milliseconds: 100));
  throw Exception('Network failed');
}

Future<void> main() async {
  try {
    final data = await loadData();
    print(data);
  } catch (e) {
    print('Handled: $e');
  } finally {
    print('Finished');
  }
}
```

Expected Output

```
Handled: Exception: Network failed
Finished
```

মনে রাখবে: Future-এ `.catchError(...)`-ও আছে। Stream listen করলে `onError` handler দিতে পারো; async error কখন handle হচ্ছে, সেটি গুরুত্বপূর্ণ।

TOPIC 06 / 10

Null safety

Dart-এ type ডিফল্টভাবে non-nullable। `String`-এ `null` রাখা যায় না; `String?`-এ যায়। Nullable value ব্যবহারের আগে null check বা নিরাপদ operator ব্যবহার করো।

Code Example

```
void main() {
  String? nickname;
  print(nickname ?? 'Guest');

  nickname = 'Mina';
  print(nickname?.length);

  if (nickname != null) {
    print(nickname.toUpperCase());
  }
}
```

Expected Output

```
Guest
4
MINA
```

মনে রাখবে: `?.` null-aware access, `??` fallback, `!` assertion। `!` কেবল নিশ্চিত হলে দেবে—null হলে runtime exception হয়। `late` initialization পিছিয়ে দেয়, তবে ভুলে ব্যবহার করলে error হতে পারে।

TOPIC 07 / 10

Exception handling

Unexpected failure handle করতে `try`, `on`, `catch`, `finally`, `throw` ব্যবহার হয়। `on` দিয়ে নির্দিষ্ট exception type ধরা যায় এবং `catch` দিয়ে exception object পাওয়া যায়।

Code Example

```
int divide(int a, int b) {
  if (b == 0) throw ArgumentError('b cannot be 0');
  return a ~/ b;
}

void main() {
  try {
    print(divide(10, 0));
  } on ArgumentError catch (e) {
    print('Invalid input: $e');
  } finally {
    print('Done');
  }
}
```

Expected Output

```
Invalid input: Invalid argument(s): b cannot be 0
Done
```

মনে রাখবে: শুধু error লুকানোর জন্য খালি `catch` ব্যবহার করো না। সম্ভব হলে নির্দিষ্ট exception handle করো; প্রয়োজন হলে `rethrow` ব্যবহার করো।

Generics

Generics হলো type parameter (যেমন `T`) ব্যবহার করে reusable কিন্তু type-safe class/function লেখা। ফলে একই logic `int`, `String` বা অন্য type-এর জন্য কাজ করে।

Code Example

```
class Box<T> {
  final T value;
  Box(this.value);
  T getValue() => value;
}

T firstItem<T>(List<T> items) => items.first;

void main() {
  final numberBox = Box<int>(42);
  print(numberBox.getValue());
  print(firstItem<String>(['A', 'B']));
}
```

Expected Output

```
42
A
```

মনে রাখবে: `List<String>`, `Map<String, int>`-ও generics-এর উদাহরণ। Constraints লাগলে `T extends SomeType` লিখতে পারো।

Enums

Enum দিয়ে নির্দিষ্ট সংখ্যক named constant প্রকাশ করা হয়—যেমন status বা direction। Dart enum-এ `switch` ব্যবহার করে প্রতিটি case আলাদা handle করা যায়। Enhanced enum-এ field/method-ও থাকতে পারে।

Code Example

```
enum OrderStatus { pending, shipped, delivered }

String label(OrderStatus status) {
  return switch (status) {
    OrderStatus.pending => 'Pending',
    OrderStatus.shipped => 'Shipped',
    OrderStatus.delivered => 'Delivered',
  };
}

void main() {
  print(label(OrderStatus.shipped));
}
```

Expected Output

```
Shipped
```

মনে রাখবে: Enum member-এর `.name`, `.index` এবং `OrderStatus.values` ব্যবহার করতে পারো।

TOPIC 10 / 10

Extensions

Extension দিয়ে বিদ্যমান type-এর উপর নতুন method/getter যোগ করা যায়, মূল class পরিবর্তন বা subclass না করেই। Extension method statically resolved হয়।

Code Example

```
extension StringTools on String {
  String capitalize() {
    if (isEmpty) return this;
    return this[0].toUpperCase() + substring(1);
  }
}

void main() {
  print('dart'.capitalize());
  print(''.capitalize());
}
```

Expected Output

```
Dart
(একটি খালি লাইন)
```

মনে রাখবে: Extension আসল class-এর private member access করতে পারে না। এটি নতুন instance field যোগ করে না।

PRACTICE

নিজে অনুশীলন করো

1. `Shape` interface তৈরি করো, এবং `Circle` class-এ `area()` implement করো।

2. `Future` দিয়ে ২ সেকেন্ড পরে একটি message return করো।

3. `async/await` দিয়ে দুটি Future-এর result print করো।

4. `Stream` থেকে ১–৫ সংখ্যাগুলো print করো।

5. Future error-কে `try/catch` দিয়ে handle করো।

6. `String?` ব্যবহার করে null হলে `Unknown` দেখাও।

7. ভুল input পেলে `ArgumentError` throw করো।

8. `Box` দিয়ে একটি String store করো।

9. `TrafficLight` enum দিয়ে তিনটি state তৈরি করো।

10. `int`-এর extension লিখে সংখ্যা even কি না বের করো।

দ্রুত তুলনা

Future: একটি asynchronous result। **Stream**: ধারাবাহিক asynchronous events। **try/catch**: synchronous ও awaited asynchronous error handle করতে পারে। **Null safety**: nullable data স্পষ্টভাবে নির্ধারণ করে।

Reference: dart.dev/language | dart.dev/libraries/async